

CacheFlow: Caching Generated Reasoning, Not Just the Prompt, for Persistent-Memory Coding Agents

Agastya Choudhary
agastya.r.choudhary@gmail.com

July 8, 2026

Abstract

Coding agents pay for the same computation twice. On *self-hosted* models the prompt prefix—a system prompt plus tens of thousands of tokens of codebase context—is re-evaluated on every invocation even though its attention (KV) state is deterministic and reusable. On *cloud-hosted* models the situation is worse: provider prompt caching amortizes the stable prefix, but *extended-thinking tokens*—the reasoning a model generates before answering—are produced fresh on every call and cannot be cached at all, because they are generated output, not cached input. In multi-agent and multi-step workflows, independent agents re-reason about the same problem and re-read the same files, paying full reasoning cost each time. We present CACHEFLOW, a persistent-memory layer that addresses both costs with mechanisms matched to what each deployment actually exposes. For self-hosted `llama.cpp` models, CACHEFLOW serializes and restores per-sequence KV state, turning a multi-second cold prefill into a sub-second warm restore. For cloud models, where no internal state is retrievable, it instead caches the model’s *outputs*: a content-addressed *thinking-block store* with confidence-gated semantic reuse, and a versioned *knowledge pool* of region summaries invalidated automatically by content hashing. Reuse is enforced—not merely advised—through a `PreToolUse` hook that intercepts redundant file reads. Across eight real-world Python corpora we measure a cold prime replaced by a ~ 300 ms warm restore, cumulatively ~ 246 s of wall-clock prefill and $\sim 1,500$ TFLOP of compute avoided on the self-hosted path, and exact, API-reported reductions of up to 100% of a turn’s reasoning tokens on the cloud path, with all savings computed from real token counts rather than estimated from text length.

1 Introduction

Large-language-model coding agents are increasingly deployed as *long-lived*, *multi-step*, *multi-agent* systems: a planner delegates to an implementer, a reviewer inspects the diff, a tester exercises the change, and each of these may run dozens of tool-calling iterations. Every one of these invocations begins by presenting the model with a large, mostly-stable context (a system prompt plus a substantial slice of the codebase) and then asks the model to reason toward an action.

Two distinct, recurring costs arise from this pattern, and—critically—they have *different* solutions depending on whether the model is self-hosted or cloud-hosted:

1. **Prefix re-evaluation (self-hosted).** A self-hosted model exposes its own attention/KV state. Yet most serving stacks discard that state between calls and re-run the full prefill over the stable prefix, spending seconds of wall-clock time and hundreds of TFLOPs re-deriving a state that is deterministic given the same tokens and model.
2. **Reasoning regeneration (cloud-hosted).** A cloud API never exposes internal state; there is no `llama_state_seq_get_data` equivalent to call. Provider-side prompt caching [5] amortizes the *input* prefix, but *extended-thinking* tokens are *generated output*. They are recomputed on every call and, once a provider’s short cache TTL lapses, cannot be shared even between two agents solving the identical

problem minutes apart.

The central observation of this paper is that these are two faces of the same waste—*recomputing a deterministic function of the same inputs*—but that they must be attacked at different layers because cloud and self-hosted deployments expose different surfaces. Where the model’s internals are reachable, cache the *internals* (KV state). Where they are not, cache the model’s *outputs* (reasoning traces and derived understanding).

CACHEFLOW implements both. It began as a KV-cache restoration system for self-hosted `llama.cpp` models (Section 5) and was extended with an output-caching layer for cloud models (Section 6). The two subsystems share design philosophy—content-hash-based staleness, snapshot/GC lifecycle, exact rather than estimated metrics—but are fully independent in storage and invalidation.

Contributions.

- A persistent per-sequence KV-cache snapshot/restore system for self-hosted models that skips the redundant warm-path save and forces a safe re-prime on codebase or model change (Section 5).
- A two-store output-caching layer for cloud models: a content-addressed, confidence-gated *thinking-block store* and a hash-invalidated *knowledge pool* (Section 6).
- An *enforcement* mechanism—a `PreToolUse` hook

that blocks redundant reads—distinguishing our approach from purely advisory prompt-engineering (Section 7).

- A discipline of *exact* token accounting: every savings figure is a real, provider-reported token count, never estimated from character length (Section 8).
- An evaluation across eight real-world Python corpora quantifying both paths, including honest reporting of zero-hit cases (Section 9).

2 Background and Motivation

2.1 The Prefill Tax on Self-Hosted Models

Transformer inference proceeds in two phases: a compute-bound *prefill* that populates the KV cache for every prompt token in parallel, and a bandwidth-bound *decode* that emits output tokens one at a time. For a coding agent whose prompt is dominated by a fixed system prompt and codebase slice, prefill is pure overhead after the first call: the KV state for those tokens is identical across invocations of the same model. A multi-thousand-token codebase prefill on an 8B model costs on the order of tens of seconds of wall-clock time and hundreds of TFLOPs in our measurements (Section 9: e.g. a cold prime of ~ 20 s on the `requests` corpus), all of it re-derivable from disk in milliseconds if the state is persisted.

2.2 The Reasoning Tax on Cloud Models

Extended thinking lets a model emit an internal reasoning trace before its final answer, materially improving performance on hard tasks. But this trace is *output*: it is billed as output tokens and regenerated on every call. Consider a single agent running a 20-step tool-use loop at $\sim 5,000$ thinking tokens per step: 100,000 reasoning tokens, none of which any provider cache can eliminate. In a multi-agent workflow the same reasoning is paid two or three times as each agent independently re-derives it. And because provider caches expire on the order of minutes, an agent that starts even slightly after another gets no benefit from the first agent’s work.

2.3 Why Existing Techniques Are Insufficient

Provider prompt caching [5] and prefix-reuse systems such as PromptCache [2] and PagedAttention [1] target the *input* prefix; they do not touch generated reasoning. Context-compaction and summarization reduce message bloat but do not stop each agent from thinking independently. Retrieval-augmented generation [3] injects relevant documents but does not cache the *conclusions* an agent reaches. CACHEFLOW’s cloud path is complementary to all of these: it caches the reasoning and the derived understanding themselves.

3 Problem Formulation

We now make the two costs of Section 1 precise and derive the condition under which caching is profitable.

Table 1: Notation.

Symbol	Meaning
P	length (tokens) of the stable prefix (system prompt + code)
s	length of the per-call task suffix, $s \ll P$
N	number of invocations sharing the same prefix/problem
c_{pre}	cost to prefill one token (compute or wall-clock)
c_{res}	cost to restore a KV snapshot from disk
T_r	extended-thinking tokens generated on a cold reasoning turn
T_v	tokens spent validating a borderline cached block
$\sigma(\cdot)$	similarity score of a candidate cached artifact
h	content hash; h_w at write time, h_r at read time
p	empirical reuse (cache-hit) rate over the N calls
θ_u, θ_v	use-directly and validate confidence thresholds

Table 1 fixes notation.

3.1 Self-Hosted: the Prefill Recurrence

Without persistence, every one of the N calls re-prefills the whole prefix, so total prefill cost is

$$C_{\text{base}} = N(P + s)c_{\text{pre}}. \quad (1)$$

With snapshot persistence the first call primes (P tokens) and each subsequent call restores the snapshot and prefills only its suffix:

$$C_{\text{cf}} = \underbrace{P c_{\text{pre}}}_{\text{one cold prime}} + (N-1)(c_{\text{res}} + s c_{\text{pre}}). \quad (2)$$

Since $c_{\text{res}} \approx 0$ relative to $P c_{\text{pre}}$ and $s \ll P$, the per-warm-call saving approaches $P c_{\text{pre}}$ and the speedup grows with prefix size:

$$\frac{C_{\text{base}}}{C_{\text{cf}}} \xrightarrow{s, c_{\text{res}} \rightarrow 0} \frac{N(P + s)}{P} \approx N. \quad (3)$$

The compute form of c_{pre} is not constant in P : a decoder layer’s attention contributes a term quadratic in sequence length. For a model of Π parameters, L layers, and hidden width d , the FLOPs avoided by skipping a prefill of n tokens is

$$\mathcal{F}(n) = \underbrace{2\Pi n}_{\text{dense (linear)}} + \underbrace{2Ldn^2}_{\text{self-attention (quadratic)}}. \quad (4)$$

We evaluate Equation (4) with Π , L , d read exactly from the loaded GGUF (Section 8), never estimated from a model-name size tag.

3.2 Cloud: Expected Reasoning Cost Under Reuse

On the cloud path there is no c_{res} to exploit; the recurring cost is the reasoning T_r regenerated per call. Let p

be the fraction of calls that find a usable cached block. A cold call costs T_r ; a validated reuse costs only T_v (the confirmation call, $T_v \ll T_r$); an exact/high-confidence reuse costs 0. Modelling the hit population as a fraction α of exact or high-confidence hits and $1 - \alpha$ validated hits, the expected per-call reasoning cost is

$$\mathbb{E}[T] = (1 - p) T_r + p[(1 - \alpha) T_v], \quad (5)$$

and the fractional reasoning saving is

$$\eta = 1 - \frac{\mathbb{E}[T]}{T_r} = p \left(1 - (1 - \alpha) \frac{T_v}{T_r} \right). \quad (6)$$

Equation (6) makes the design tension explicit: savings rise with hit rate p , but a reused block that is *wrong* corrupts the downstream call, so p must be earned conservatively rather than maximised (Section 6.1).

3.3 The Reuse Decision

Retrieval returns the best candidate and a calibrated confidence $\kappa \in [0, 1]$; the action is a thresholded decision

$$a(\kappa) = \begin{cases} \text{USE} & \kappa \geq \theta_u, \\ \text{VALIDATE} & \theta_v \leq \kappa < \theta_u, \\ \text{RE_THINK} & \kappa < \theta_v. \end{cases} \quad (7)$$

Confidence combines raw similarity with an age penalty; for a block written t days ago,

$$\kappa = \sigma(\cdot) \cdot e^{-\lambda t}, \quad \lambda = 0.1, \quad (8)$$

a ~ 7 -day half-life reflecting that older reasoning was computed against a codebase that has since drifted. When the pool holds ≥ 8 blocks we replace the absolute σ with a scale-invariant background z -score

$$z = \frac{\sigma^* - \mu_\sigma}{\varsigma_\sigma}, \quad (9)$$

where σ^* is the top candidate’s similarity and $\mu_\sigma, \varsigma_\sigma$ are the mean and standard deviation of the query’s similarity to the rest of the pool—measuring how far the best hit *stands out*, which transfers across domains where the absolute scale of σ does not.

3.4 Staleness as a Hash Predicate

Both subsystems replace explicit invalidation with a write-time/read-time hash comparison. An artifact is served iff

$$\text{VALID} \iff h_r = h_w, \quad (10)$$

so any edit to the underlying prefix, file region, or model silently changes h_r and suppresses the stale artifact with no separate expiry path.

4 System Overview

CACHEFLOW is a command-line tool (`cf`) and a set of libraries packaged as a single Python distribution. It comprises two independent subsystems joined by a shared philosophy but no shared state:

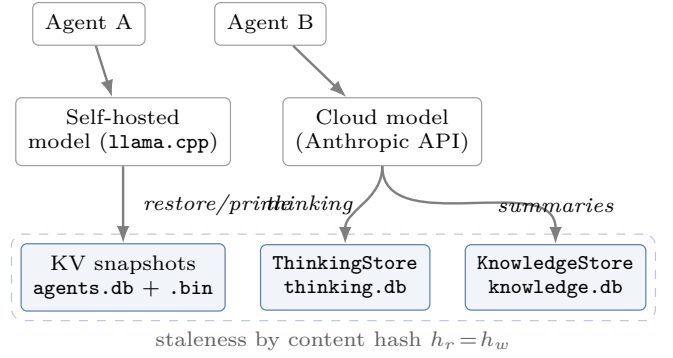


Figure 1: CACHEFLOW’s two independent subsystems. Where model internals are reachable (self-hosted), it caches KV state; where they are not (cloud), it caches generated *outputs* in two pools. No store shares keys with another.

Self-hosted KV engine

Serializes and restores per-sequence attention state for local `llama.cpp` models (Section 5).

Cloud output cache

Two SQLite-backed pools—`ThinkingStore` and `KnowledgeStore`—that cache generated reasoning and derived file summaries for cloud-hosted agents (Section 6).

Both subsystems detect staleness the same way: a content hash computed at *write* time is compared at *read* time (Equation (10)), so a changed codebase, file, or model silently invalidates the cached artifact with no explicit expiry logic. They store to distinct SQLite databases (`.cacheflow/agents.db`, `thinking.db`, `knowledge.db`) and share no foreign keys: a KV-snapshot restore and a thinking-block hit are unrelated events that can each occur independently (Figure 1).

5 The Self-Hosted KV-Cache Engine

5.1 Core Flow: Restore/Prime $\rightarrow$ Save $\rightarrow$ Complete $\rightarrow$ Record

Each agent maintains a HEAD pointer to at most one KV snapshot. A session proceeds as:

- 1. Restore or Prime.** The agent computes `stable_context` (system prompt + codebase text) and its hash. If the agent’s HEAD snapshot’s `stable_context_hash` still matches, the snapshot is restored (*warm path*). Otherwise the prefix is fed to the model to populate the KV cache (*cold prime*).
- 2. Save.** On the prime path *only*, the KV cache is serialized to disk. On the warm path this is skipped: the HEAD snapshot on disk is already byte-identical.
- 3. Complete.** The model generates using prefix matching—cached tokens plus the short (~ 5 –50 token) task suffix.
- 4. Record.** The agent’s HEAD and token/time metrics

Algorithm 1: Restore-or-Prime session (self-hosted)

Input: agent A , stable context X , active model M

```
1  $h_r \leftarrow \text{HASH}(X)$ 
2 if  $A.\text{head} \neq \emptyset$  and  $A.\text{head}.h_w = h_r$  and
    $A.\text{head}.model = M$  then
3    $kv \leftarrow \text{RESTORESNAPSHOT}(A.\text{head})$  // warm:  $O(c_{\text{res}})$ 
   // skip save: on-disk HEAD already
   // byte-identical
4 else
5    $kv \leftarrow \text{PREFILL}(X)$  // cold:  $O(P_{\text{cpre}})$ 
6    $\text{snap} \leftarrow \text{SERIALIZESEQ}(kv)$ 
7    $\text{WRITESNAPSHOT}(\text{snap}); A.\text{head} \leftarrow \text{snap}$ 
   // write-then-update HEAD
8  $y \leftarrow \text{COMPLETE}(kv, \text{suffix})$ 
9  $\text{RECORDMETRICS}(A); \text{return } y$ 
```

are updated in SQLite; savings are computed against the cold-prime baseline via Equations (1) and (2).

Algorithm 1 makes the two safety invariants explicit: the warm branch is taken only when *both* the content hash and the model match (a snapshot is raw KV bytes tied to one tokenizer/vocabulary), and the snapshot is written *before* HEAD is advanced, so a crash leaves A on its previous valid snapshot.

5.2 Per-Sequence Snapshots (Format v4)

Rather than serialize the full `n_ctx` buffer, CACHEFLOW saves only the live KV for the sequence via `llama_state_seq_get_data`, yielding compact snapshots ($\sim 8\text{MB}$ of in-memory slot state per idle agent). Because a snapshot is raw KV bytes tied to one model’s tokenizer, vocabulary, and hidden dimensions, a `model_changed` guard in `AgentSession` forces a re-prime (never a restore) whenever the active model differs from the one that produced the snapshot.

5.3 Concurrency, Consolidation, and GC

A `SlotPool` manages up to eight concurrent KV slots with a `SlotLease` context manager and LRU eviction that never evicts an actively-running agent. Past 70% of the context window, a background `Compressor` restores HEAD, asks the model for a ≤ 500 -token knowledge summary, and folds it into the next session’s stable prefix. A `SnapshotGC` reaps snapshot files unreferenced by any agent’s HEAD, plus crash orphans; the write-snapshot-then-update-HEAD ordering guarantees a crash leaves the agent on its previous valid snapshot.

5.4 In-Process Execution

The model runs in-process via `llama-cpp-python` rather than behind an HTTP shim. On macOS an HTTP transport throttles decode by roughly $10\times$; in-process execution avoids that path. Decode remains bandwidth-bound at $\sim 5\text{tokens s}^{-1}$, so the engine optimizes the compute-bound prefill (`n_batch=n_ubatch=2,048`) where caching pays off, and a single global model instance (`get_global_engine`) is shared across all agents.

The prefill batch width is a deliberate choice. `llama.cpp` decodes a prompt in chunks of `n_batch` tokens; the default 512 forces a multi-thousand-token codebase prefix through many sequential chunks. Raising `n_batch=n_ubatch` to 2,048 lets the same prefix populate in a quarter as many passes, and the engine enables FlashAttention so that a stable context spanning several batch chunks is attended to in a single fused kernel with $O(d)$ rather than $O(n)$ intermediate memory per token—a prerequisite for priming real-codebase contexts that exceed one batch width. A one-time `atexit` tear-down then frees the inference model and the vocabulary-only tokenizer model (Section 8) in a fixed order, since both hold buffers in the same `ggml` Metal device manager and must be released deterministically rather than at interpreter-shutdown GC order.

5.5 Per-Model Instruction Templating

A KV snapshot is only reusable if every session wraps its turns in the byte-exact token markers the model was trained on; a prefix that drifts by a single control token no longer matches the cached KV. CACHEFLOW therefore selects an instruction template per model rather than hard-coding one family. A `ChatTemplate` record fixes the system/user wrappers, the assistant open/close markers, and the generation stop token for each of five families (CHATML, LLAMA 3, MISTRAL, GEMMA, PHI 3). `detect_template` resolves the family by a three-stage cascade, ordered by reliability:

1. **Metadata sniff.** If the GGUF’s embedded `tokenizer.chat_template` (exposed as `Llama.metadata`) contains a distinctive marker—`<|start_header_id|>`, `[INST]`, `<start_of_turn>`—the family is read straight from the model file.
2. **Name match.** Otherwise the model name is matched against known family strings.
3. **Default.** Failing both, CHATML is assumed.

Families with no dedicated system role (MISTRAL, GEMMA; `supports_system=FALSE`) fold the system prompt into the first user turn rather than dropping it, and the agentic loop’s generation `stop` list uses the family’s own `stop_token` so multi-turn assistant boundaries terminate correctly on every model.

5.6 The Agentic Loop

`cf agent` drives an observe-act loop (`run_agentic`) that is deliberately decoupled from `AgentSession`: it touches only the KV-cache-facing primitives an external harness would have (`_restore_or_prime`, `server.completion`, lock acquire/release). The codebase KV stays resident in one slot for the whole loop, so every step prefix-matches the cached prefix and evaluates only the new observation plus the generated action. Each step emits a `THOUGHT/ACTION/ARGS` triple parsed and dispatched against a tool registry (`read_file`, `edit_file`, `write_file`, `grep`, `run_bash`, `finish`), extended with native pool tools (Section 6).

Two termination conditions bound the loop beyond `max_steps`: (i) if a completion is byte-identical to the previous one twice consecutively—the signature of deterministic decoding re-emitting the same malformed action against identical error feedback—the loop halts with a `stuck_loop` step; (ii) before each completion, if the prompt’s token count plus `max_tokens_per_step` would meet or exceed `ctx_size`, it halts with a `context_limit` step. Both return the partial trajectory rather than raising, so a stalled model degrades to a truncated result instead of a crash.

5.7 Sandboxed Execution

Because an agentic run issues unsupervised edits and shell commands, `cf agent -auto/-allow-bash` executes inside a `GitWorktreeSandbox` by default. CACHEFLOW uses a `git worktree` rather than a container: a worktree shares the repository’s object store, so it is created in milliseconds with no blob copying, and `git` is already a hard dependency (source files are enumerated with `git ls-files`). The flow (Figure 2) creates a worktree off HEAD on a throwaway `cacheflow/sandbox-*` branch, runs every tool call inside it while the KV-cache config/store/snapshots at `base_path` stay untouched, commits whatever changed, optionally runs a test command, and merges back *only* if there is something to merge and (when given) the tests pass. On test failure or merge conflict the sandbox branch is left in place for inspection rather than discarded. Entry requires a clean working tree—excluding `.cacheflow/` via a `!.cacheflow` pathspec, since the engine’s own SQLite/WAL files churn every session—and raises rather than guessing how to reconcile uncommitted work.

6 The Cloud Output-Cache Layer

Because a cloud API exposes no internal state, this layer caches the model’s *outputs*. It comprises two independent stores.

6.1 Tier 1: The Thinking-Block Store

When an agent completes an extended-thinking turn, the block is captured and submitted to `ThinkingStore.submit`, which persists it to SQLite together with a problem hash, a codebase hash, and—when `sentence-transformers` is available—a sentence embedding of the *task description* (stored as a pickled BLOB on the row). The embedding model is `all-MiniLM-L6-v2` [4] (~90 MB, 384-dimensional). This is a deliberate downgrade: an earlier version used `intfloat/e5-mistral-7b-instruct`, a 7B, ~13 GB model whose load swap-stormed and hard-froze the machine; the small MiniLM model loads safely anywhere, and setting `CACHEFLOW_DISABLE_EMBEDDINGS=1` skips embedding entirely and reverts to exact-hash-only matching.

Retrieval is a cascade (Table 4):

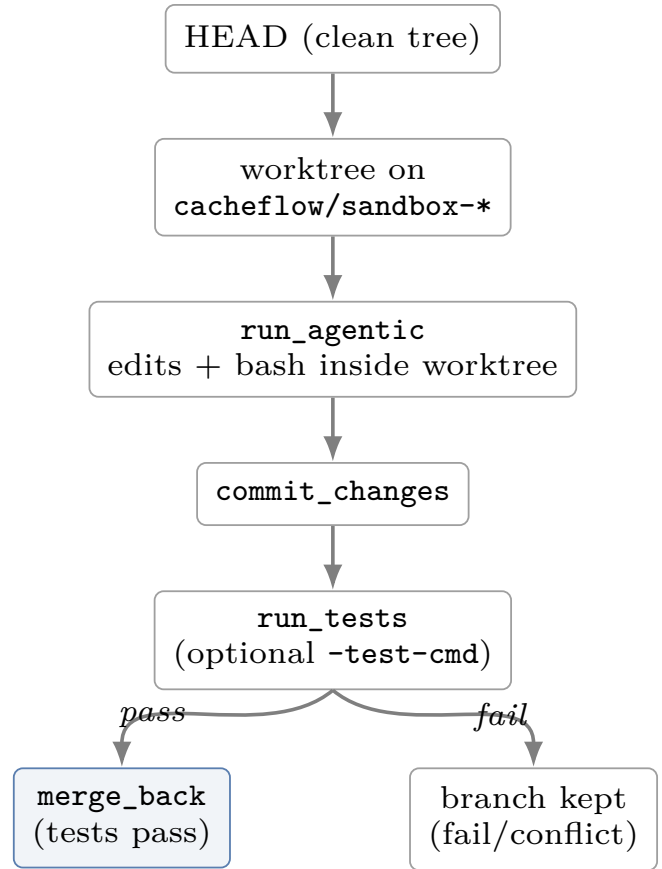


Figure 2: Sandboxed agentic execution. All edits land in a git worktree sharing the object store; the real tree is modified only by `merge_back`, gated on optional tests.

1. **Exact hash lookup** (<1 ms): a SHA-256 match on the problem hash returns the block immediately.
2. **Semantic search** (~30–50 ms to embed on CPU): on a miss, the query is embedded and scored against the stored embedding BLOBs by cosine similarity. The primary path is an in-process scan in SQLite—no external vector service is required (an optional Qdrant index is used only if one happens to be reachable). Each raw similarity is multiplied by an age-decay weight $\exp(-0.1 t_{\text{days}})$ (~7-day half-life), since an older block was computed against a codebase that has likely drifted further from the current state.
3. **Confidence gating**. The weighted score selects an action. The thresholds are calibrated to this model *and* this domain, not borrowed from paraphrase benchmarks: for long, multi-clause engineering-task rewordings, MiniLM cosine sits in a ~0.45–0.84 band while genuinely unrelated tasks top out near 0.46. Absolute-cosine gates are therefore $\geq 0.62 \Rightarrow \text{use_directly}$, $\geq 0.40 \Rightarrow \text{validate}$, else `re_think`. When the store holds ≥ 8 blocks—enough for a stable mean/variance—the system prefers a scale-invariant *background z-score* (how far the candidate stands out from the query’s similarity to the rest of the pool),

Algorithm 2: Thinking-block retrieval cascade

Input: task q , pool $\mathcal{B}$, thresholds θ_u, θ_v
Output: (block, action)

```

1 if  $\exists b \in \mathcal{B} : b.h = \text{HASH}(q)$  then return  $(b, \text{USE})$  // <1 ms
2 if no embedding model then return  $(\emptyset, \text{RE\_THINK})$ 
3  $v \leftarrow \text{EMBED}(q)$  // MiniLM, 384-d, ~40 ms
4  $b^*, \sigma^* \leftarrow \arg \max_{b \in \mathcal{B}} \cos(v, b.\text{emb})$ 
5  $\kappa \leftarrow \sigma^* \cdot e^{-\lambda t_{b^*}}$  // age decay, Eq. 8
6 if  $|\mathcal{B}| \geq 8$  then
7    $\kappa \leftarrow z\text{-score}(\sigma^*; \mathcal{B})$  // scale-invariant, Eq. 9
8    $(\theta_u, \theta_v) \leftarrow (4.0, 2.8)$ 
9 if  $\kappa \geq \theta_u$  then
10 | LOGREUSE( $b^*$ ); return  $(b^*, \text{USE})$ 
11 else if  $\kappa \geq \theta_v$  then
12 | stash  $b^*$ ; return  $(b^*, \text{VALIDATE})$  // no log yet
13 else return  $(\emptyset, \text{RE\_THINK})$ 
```

gating at $z \geq 4.0$ (use) and $z \geq 2.8$ (validate).

The three-way action is deliberate: semantic similarity is not certainty, and naively reusing any “close enough” block risks propagating a wrong line of reasoning into a new problem. The **validate** tier is the middle ground between “always re-think” (no savings) and “always reuse” (correctness risk); a validate-band candidate is not counted as a saving until an external check (in our runner, a cheap Claude Haiku validator) confirms it via `commit_validated_reuse`—retrieval alone never logs a reuse.

Every confirmed hit logs the matched block’s real `token_count` into a `thinking_reuse_log` table, so `cf thinking stats` can report *exact* cumulative tokens saved. The store degrades gracefully: with no embedding model available, it falls back to exact-hash-only matching rather than raising.

Code-change robustness (the reuse delta). Semantic similarity of *task descriptions* says nothing about whether the *code* the cached reasoning was computed against still holds. Each block therefore carries a **delta**: the set of files, functions, and line ranges its reasoning depended on, recorded at submission from the git working-tree diff. On retrieval the candidate’s **delta** is intersected with the files changed since it was stored; any overlap discounts confidence toward the **validate** band. Writing Δ_b for a block’s dependency set and C for the currently-changed files, the gated confidence is

$$\hat{\kappa} = \kappa \cdot e^{-\lambda t_b} \cdot (1 - \mathbb{P}[\Delta_b \cap C \neq \emptyset] \rho), \quad (11)$$

with overlap penalty ρ pulling an otherwise-strong match down into **validate** rather than **use_directly**. This makes reuse robust to incidental edits—an unrelated file changing does not invalidate the block—while refusing to splice reasoning that a relevant edit may have invalidated.

Problem-type routing and roles. Submission tags each block with a coarse `problem_type` from

`classify_task`—a keyword heuristic bucketing tasks into `{implement, review, debug, refactor, test}`—and an optional role (`implementer, reviewer, tester`). These are not precision instruments; they bucket similar problems so that a reviewer’s reasoning is preferentially matched to another review task, and retrieval falls back to a role-agnostic match when no same-role block exists. The knowledge pool applies the identical role-then-generic fallback.

6.2 Tier 2: The Knowledge Pool

For work that does not use extended thinking, `KnowledgeStore` caches dense per-region summaries. `submit` stores a summary for a file/region path and marks any prior entry for the same (`region, role`) as superseded (versioned via `supersedes_id`, not deleted). `query` returns a summary *only* if the caller’s current `region_hash` matches the stored hash—staleness is automatic, with no separate invalidation path—and falls back from a role-specific match to a generic (`role IS NULL`) entry. The region hash follows the `git hash-object` convention so it is identical whether computed by the skill, the CLI, or the enforcement hook.

6.3 Capture via Transcript Parsing

Submission is automated by a `PostToolUse` hook. `extract_thinking_blocks_from_transcript` reads the Claude Code session transcript (JSONL), finds the most recent assistant turn containing **thinking** blocks, and pairs it with the nearest preceding user turn’s text as the problem description. Crucially, it hashes the task description *alone*, matching `query`’s hashing convention—an earlier version folded the codebase hash into the key, which meant nothing the hook captured could ever be retrieved. The hook is best-effort by construction: malformed or missing transcripts yield an empty list rather than raising, because a hook that breaks the agent it is attached to is worse than a missed cache opportunity.

6.4 Storage Schema

Both pools are plain SQLite, one file each, with no cross-table foreign keys (Listing 1). `thinking_blocks` holds the reasoning text, its embedding BLOB, the two content hashes, the **delta**, and the routing tags; `thinking_reuse_log` appends one row per confirmed reuse carrying the exact tokens saved, which is what `cf thinking stats` sums. `knowledge_entries` versions summaries via `supersedes_id`: a new summary for a (`region, role`) pair marks the prior row superseded rather than deleting it, so history is auditable and GC is explicit.

Listing 1: Pool schemas (abridged): two independent SQLite files.

```
-- thinking.db
thinking_blocks(
  id, thinking_block TEXT, embedding BLOB, -- 384-d
```

```

MiniLM, pickled
problem_hash TEXT, codebase_hash TEXT,
delta JSON,                                -- files/
    fns/lines depended on
problem_type TEXT, role TEXT,
token_count INTEGER,                       -- exact;
    NULL if unknown
created_at, accessed_at, ...)
thinking_reuse_log(
    id, thinking_block_id, tokens_saved INTEGER, reused_at
)
-- knowledge.db
knowledge_entries(
    id, region TEXT, region_hash TEXT,      -- git hash
    -object
    role TEXT, summary TEXT, token_count INTEGER,
    supersedes_id INTEGER, created_at)

```

7 Enforcement, Not Advice

A recurring weakness of prompt-level optimizations is that they are advisory: the agent is *told* to check a cache before reading, but can simply not do so. CACHEFLOW makes the knowledge pool the one part of the cloud system that is *enforced*. Setup is one command: `cf install` renders a single canonical skill body—“consult the pool before reading, submit a summary after working”—into each supported harness’s own format (`.claude/skills/`, `.cursor/rules/*.mdc`, `.codex/`) and is idempotent by content comparison, so a target already matching the rendered body is reported *unchanged*. It then registers two hooks in `.claude/settings.json`, scanning existing entries first so re-running never double-registers: a `PostToolUse` capture hook (Section 6.1) and the enforcement hook below. The skill is advice the model may ignore; the enforcement hook is not. `cf install` registers `cf knowledge check-before-read` as a `PreToolUse` hook matched on the harness’s `Read` tool. Before every read, the hook parses the tool-input payload, hashes the target file’s current contents, and queries the knowledge pool. On an exact-hash hit it writes a redirect message to `stderr` and exits with code 2, which the harness surfaces to the model as feedback it must act on before the read proceeds. On any ambiguity (non-`Read` tool, missing payload, no git, no cached summary) it exits 0 and allows the read: the hook holds an informed opinion only on an exact hit, never a guess. Listing 2 shows the core of the entry point.

Listing 2: The `PreToolUse` enforcement hook (elided). An exact-hash hit blocks the read via exit code 2; every ambiguous case falls through to allow it.

```

def knowledge_check_before_read():
    try:
        ev = json.load(sys.stdin)
        if ev.get("tool_name") != "Read":
            return # exit 0: not our concern
        path = ev["tool_input"]["file_path"]
        h = compute_region_hash(path) # git hash-
        object
        hit = KnowledgeStore().query(path,

```

```

current_region_hash=h)
    if hit:                                # fresh,
        exact-hash summary exists
        sys.stderr.write(
            f"Cached_summary_for_{path}_is_current;_
"
            f"use_'cf_knowledge_query_'_instead_of_re
-reading.")
        sys.exit(2)                        #
    SystemExit: NOT caught below
except Exception:
    return # any failure -> allow the read, never
    block on an error

```

This exposes a design boundary. The enforcement hook is harness-specific—it intercepts Claude Code’s own `Read`. For CACHEFLOW’s own local agentic loop, which has no external harness to interpose, the same benefit is delivered by giving the loop *native* tools—`knowledge_query`, `knowledge_submit`, `thinking_query`, and `cache_flow_status`—that call the stores in-process, plus preamble guidance to consult them before reasoning and submit a summary after a unit of work.

7.1 Hook Failures Are Silent by Design

The capture hook wraps its entire body in a bare `except Exception: pass`; a broken hook must no-op, not break the run. The enforcement hook follows the same rule for everything *except* its deliberate exit-code-2 block: because `SystemExit` is not an `Exception` subclass, its `except Exception: return` never swallows the intentional block, but any unrelated failure allows the read through rather than blocking on an error it has no opinion about.

8 Exact Token Accounting

A guiding principle across both subsystems is that *no savings figure is ever estimated from text length*. The `token_count` on both `ThinkingStore.submit` and `KnowledgeStore.submit` must be the real cost—e.g. the Anthropic API’s own `usage.output_tokens` for the turn that produced the block. An earlier implementation stored `len(text)` as a stand-in; a character count is not a token count and would have made every downstream figure fictional. When the caller lacks an exact number, the column is left `NULL` (both columns are nullable for this reason) rather than guessed, and `get_reuse_stats` *excludes* `NULL` rows from its sum instead of treating them as zero. Correspondingly, transcript capture attributes `output_tokens` to a block only when its turn produced *exactly one* thinking block; splitting a single turn-level total across several blocks would itself be a guess.

9 Evaluation

Setup. We evaluate on eight workloads (W1–W8), each a distinct real-world Python corpus—`itsdangerous`, `click`, `requests`, `httpx`, `flask`,

`pytest`, `sqlalchemy`, and `django`. Each workload drives multiple agent sessions against its corpus; the first session populates the relevant cache and later sessions attempt reuse. On the cloud path, every session is an *autonomous build directive*—“implement a resilient HTTP client wrapper with pooling and timeout enforcement,” not a question about the code—issued at high thinking effort. A workload’s cold session derives deep understanding of one subsystem; its warm sessions are *different* build tasks that genuinely require the same understanding (the realistic reuse case: nobody re-asks the same question, but successive features routinely need the same hard-won reasoning about the same machinery). Several workloads also plant a deliberately *unrelated* task that should miss, so the sweep measures the gate’s selectivity, not just its recall. The self-hosted path runs Qwen3-8B (36 layers, 4096 hidden dim) at an 8,192-token context via `llama.cpp`. The cloud path runs `claude-opus-4-8` through the Anthropic API. All cloud token counts are exact, read from the `usage` object. Raw per-workload results and rollups are archived under `experiments/results/`.

9.1 Self-Hosted: Prefill Avoided

Table 2 reports the KV-cache sweep. “Cache Hits” is the fraction of sessions served by a warm restore rather than a cold prime. Where a corpus admits reuse, restore latency stays in the hundreds-of-milliseconds range against multi-second (and, for the heavy `click` prime, multi-hundred-second) prime costs. W1 is single-session by construction—it establishes the cold-prime baseline and therefore shows no savings.

Across the workloads that admit reuse, `CACHEFLOW` avoids $\approx 245,731$ ms (~ 4.1 min) of cumulative wall-clock prefill and $\approx 1,513$ TFLOP of compute. W6 is instructive: it registers a cache hit and saves 4,632 tokens of prefill, yet reports *zero* wall-clock time saved because on that run the restore happened to be slower than the prime it replaced—an honest artifact of disk/scheduler variance, not a discarded saving. The FLOPs figure is computed, not guessed: the parameter count is read exactly off the loaded GGUF via `llama_model_n_params`, and the model’s `n_layer/n_embd` supply the attention term that scales with prefill length squared—for a 9,064-token prefill that term alone is ~ 16.5 TFLOP atop a ~ 127 TFLOP parameter-only floor. Output tokens are identical either way; caching eliminates prefill re-evaluation, not generation.

9.2 Cloud: Reasoning Reused Across Different Builds

Table 3 reports the cloud sweep. “Cold Calls” are sessions that reasoned from scratch (a miss, a rejected validation, or the priming first run); “Thinking Hits” are sessions served a cached block; “Baseline Reasoning” is the thinking tokens the workload’s cold calls actually generated. “Tokens Avoided” credits a hit with the *full*

derivation cost the reuse prevented—the input, reasoning, and output tokens of the cold call the warm session would otherwise have had to repeat—which is the figure that scales with corpus size, not just the reasoning sliver. The knowledge pool (Section 6.2) is exercised through the enforcement hook of Section 7 and is deliberately excluded from this token evaluation; an earlier version of the sweep staged a submit-then-query consolidation demo for it, which we removed as methodologically circular.

Across the sweep, 12 of 24 sessions (50%) were served from the thinking store, saving 5,981 reasoning tokens and 232,050 total derivation tokens. Three findings stand out. **(1) Different tasks, same reasoning, high recall.** Every warm session is a *different* build than the cold one it reuses—W7’s cold session derives `SQLAlchemy`’s session/transaction internals for a repository/unit-of-work layer, and its three hits reuse that understanding for optimistic concurrency, multi-tenant session scoping, and savepoint-based nesting (confidences 0.71–0.80, all above the 0.62 use-directly gate); W8’s cold session builds a multi-tenant billing app on `Django` and its hit reuses the tenant-isolation/model machinery for a pricing feature at confidence 0.87. W4’s three *concurrent* agents all hit (0.79–0.91), modeling teammates who pick up related work simultaneously. **(2) The gate refuses unrelated work.** The planted misses missed: W3’s streaming-multipart-upload task and W7’s encrypted-JSON column type were re-thought or validate-rejected (logged explicitly as `_miss_cold/_validate_rejected_cold`) rather than being spliced with timeout or transaction reasoning that did not apply, and W2’s warm candidate was likewise rejected by the validator—an honest sub-100% hit rate is the intended behavior of a system that prioritizes correctness over blind savings. **(3) Prompt caching and reasoning reuse compose.** Even sessions with no thinking hit were served tens of thousands of prompt-cache input tokens (73,243 across the sweep), confirming the two mechanisms are complementary: provider caching amortizes the stable *input* prefix, while the thinking store eliminates regenerated *output*—the cost provider caching cannot touch.

9.3 Latency Budget

Table 4 summarizes the reuse-path latencies that make the cloud layer worthwhile: an exact hit is effectively free, and even a validated semantic hit returns in a fraction of a second against the multi-second cost of re-reasoning.

10 Discussion and Limitations

Reuse depends on workload structure. The cloud gains concentrate where successive builds genuinely share underlying reasoning (W3, W4, and W7 each landed three hits); a task whose reasoning truly does not recur stays cold—W2’s warm candidate was validator-

Table 2: Self-hosted KV-cache engine, latest run per workload (Qwen3-8B, 8,192 ctx). Each row is the most recent `W*_local_*.json` under `experiments/results/`. TFLOPs avoided combines the $2 \times \text{params} \times \text{tokens}$ floor with the quadratic self-attention term.

W	Corpus	Sess.	Cache Hits	Prime (ms)	Restore (ms)	Time Saved (ms)	Tokens Saved	TFLOPs
W1	itsdangerous	1	0 (0%)	19,319	0	0	0	0.00
W2	click	2	1 (50%)	19,371	282	19,089	4,369	77.20
W3	requests	6	5 (83%)	19,962	353	98,044	23,894	425.09
W4	httpx	6	3 (50%)	12,630	10,688	7,885	9,033	155.99
W5	flask	2	1 (50%)	20,167	380	19,787	4,425	78.26
W6	pytest	2	1 (50%)	20,698	25,977	0	4,632	82.21
W7	sqlalchemy	10	8 (80%)	9,640	2,116	60,189	30,337	530.89
W8	django	3	2 (67%)	20,646	278	40,737	9,207	163.32
Total						245,731	85,897	1512.96

Table 3: Cloud output-cache layer, latest run per workload (claude-opus-4-8, high thinking effort, exact usage counts). Each row is the most recent `W*_cloud_*.json` under `experiments/results/`. “Cache Read” is prompt-cache input tokens served from Anthropic’s cache (`cache_read_input_tokens`)—prefill avoided even on turns with no thinking-block hit.

W	Corpus	Sess.	Cold Calls	Thinking Hits	Baseline Reason.	Reason. Saved	Tokens Avoided	Cache Read
W1	itsdangerous	1	1	0 (0%)	502	0	0	0
W2	click	2	2	0 (0%)	925	0	0	13,546
W3	requests	5	2	3 (60%)	1,083	1,329	57,603	14,989
W4	httpx	4	1	3 (75%)	949	2,847	59,097	0
W5	flask	3	2	1 (33%)	941	371	18,110	14,095
W6	pytest	2	1	1 (50%)	258	258	19,119	0
W7	sqlalchemy	5	2	3 (60%)	4,274	534	58,650	15,353
W8	django	2	1	1 (50%)	642	642	19,471	15,260
Total					9,574	5,981	232,050	73,243

Table 4: Cloud reuse-path latency budget.

Path	Latency	Note
Exact hash hit	<1 ms	no embedding
Semantic hit (high conf)	50–70 ms	embed + search
Validated hit	200–500 ms	100-token call
Re-think (miss)	~5,000 ms	full reasoning

rejected, and both planted unrelated tasks missed—correctly, since forcing reuse there would risk correctness. The knowledge pool targets a complementary regime, where reasoning does not recur but file understanding does; we exercise it through the enforcement hook of Section 7 rather than through the token sweep.

Semantic reuse is only as good as its embeddings. Confidence gating mitigates but does not eliminate the risk of a plausible-but-wrong match; the `validate` tier trades ~ 100 tokens to bound it. Absent `sentence-transformers` (or with `CACHEFLOW_DISABLE_EMBEDDINGS=1`), the system safely degrades to exact-hash matching only.

Enforcement is harness-bound. The `PreToolUse` block works only inside a harness whose read tool can be intercepted; other harnesses fall back to advisory skill

files, and the native-tool loop relies on the model choosing to call the pool tools.

The two subsystems do not compose. A KV snapshot and a thinking block invalidate on different hashes (`stable_context_hash` vs. `codebase_hash/region_hash`) for different reasons; “no thinking hit” must not be conflated with “no KV snapshot.”

11 Related Work

CACHEFLOW’s self-hosted path is closest to KV-cache reuse systems such as PagedAttention/vLLM [1] and modular prefix reuse in PromptCache [2], but targets *cross-invocation persistence* to disk for long-lived agents rather than intra-server memory management. Provider prompt caching [5] amortizes input prefixes but not generated reasoning—the exact gap our cloud path fills. Retrieval-augmented generation [3] injects source documents; we instead cache the *reasoning and conclusions* derived from them, using compact sentence embeddings [4] scored by an in-process cosine scan for semantic retrieval. To our knowledge, caching *extended-thinking output* with confidence-gated, content-hash-invalidated reuse across independent cloud agents is novel.

12 Conclusion

Coding agents recompute deterministic functions of the same inputs—prefill on self-hosted models, reasoning on cloud models. CACHEFLOW attacks both by caching at the layer each deployment actually exposes: KV state where internals are reachable, generated outputs where they are not. Measured across eight real corpora, it avoids minutes of cumulative prefill and over a teraflop $\times 10^3$ of compute on the self-hosted path, and eliminates up to 100% of a turn’s reasoning tokens—counted exactly—on the cloud path, while enforcing rather than merely advising reuse. The design’s discipline is its honesty: exact token accounting, automatic hash-based staleness, confidence-gated reuse that declines a poor match, and hooks that fail silently rather than breaking the agent they serve.

References

- [1] W. Kwon, Z. Li, S. Zhuang, et al. Efficient Memory Management for Large Language Model Serving with PageAttention. In *Proc. SOSP*, 2023.
- [2] I. Gim, G. Chen, S. Lee, et al. Prompt Cache: Modular Attention Reuse for Low-Latency Inference. In *Proc. MLSys*, 2024.
- [3] P. Lewis, E. Perez, A. Piktus, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [4] N. Reimers and I. Gurevych. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proc. EMNLP*, 2019. Model: `all-MiniLM-L6-v2`, <https://www.sbert.net>.
- [5] Anthropic. Prompt Caching with Claude. Anthropic API Documentation, 2024. <https://docs.anthropic.com/en/docs/build-with-claude/prompt-caching>.

A Workload Definitions

The eight evaluation workloads (Table 5) are real open-source Python corpora cloned into `experiments/corpora/`. They are ordered as a *difficulty ladder*—from `itsdangerous` (~1.5K LOC) to `django` (~280K LOC)—that simultaneously varies the *interaction pattern*, so the sweep exercises each distinct code path of the system rather than only scaling one axis. Every corpus is primed with the same system prompt and the RAG-seeded stable context described in Section 5; the self-hosted path runs Qwen3-8B at an 8,192-token context, the cloud path runs `claude-opus-4-8`.

Table 5: The W1–W8 workloads: a corpus difficulty ladder crossed with an interaction-pattern ladder. LOC is approximate.

W	Corpus	LOC	Interaction pattern exercised
W1	itsdangerous	1,500	Single-agent <i>cold prime</i> only—establishes the smallest cold-prime baseline; no restore.
W2	click	8,000	Single-agent cold prime <i>followed by one warm restore</i> —the minimal cache-hit case.
W3	requests	12,000	<i>Repeated warm restores</i> across five sessions—savings compound; the paper’s headline self-hosted example.
W4	httpx	18,000	<i>Concurrent multi-agent</i> : three agents on three slots, exercising the <code>SlotPool</code> and cooperative KV swap.
W5	flask	15,000	<i>Multi-step agentic loop</i> with tool use (<code>run_agentic</code>), exercising Section 5.6.
W6	pytest	40,000	<i>Fork</i> parent → child; the child starts warm from the parent’s inherited HEAD snapshot.
W7	sqlalchemy	80,000	<i>Consolidation trigger</i> (self-hosted): the accumulator crosses 70% of context, invoking the background <code>Compressor</code> .
W8	django	280,000	<i>Largest realistic prefix</i> : end-to-end stress on the biggest codebase context.

On the cloud path each workload maps the same corpora onto autonomous build directives instead of restore patterns: W1 is a single cold build (a password-reset token flow on `itsdangerous`); W2 builds a multi-command deploy CLI cold, then attempts a different build (shell autocompletion) warm; W3 builds a resilient HTTP client wrapper cold, then issues three differently-worded hardening builds that need the same timeout/pooling reasoning, plus one unrelated task (streaming multipart upload) planted to miss; W4 primes an error-handling refactor and fans out three *concurrent* related builds; W5 is a multi-step build (observability, then instrumentation reusing the lifecycle understanding, then genuinely new rate-limiting work that stays cold); W6 forks—agent A builds one pytest plugin, agent B builds a *different* plugin against the same hook/fixture machinery; W7 derives SQLAlchemy’s session/transaction internals cold, reuses them for three different builds (optimistic concurrency, multi-tenant scoping, savepoint nesting), and plants an unrelated column-type task to miss; W8 builds a multi-tenant billing app cold and a pricing-tier feature warm on `django`.

The two evaluation tables in Section 9 read the *latest* run per workload from `experiments/results/` (`W*_local_*.json` for the KV engine, `W*_cloud_*.json` for the output cache), never the earlier full-sweep rollups, which contain superseded runs.